

Trabajo Práctico N° 8

Lenguajes Orientados a Objetos

Lectura: Scott Cap. 8.3, Sebesta Cap. 9 y 12

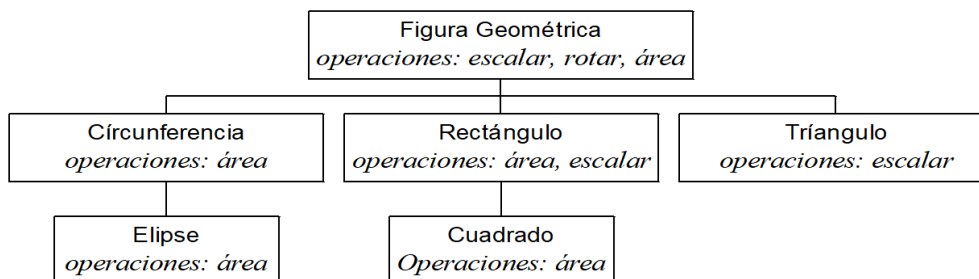
Nota: Pueden consultar el siguiente tutorial (enlace) de GNU-Smalltalk y el intérprete disponible en el siguiente enlace (enlace) para resolver esta parte del práctico.

Conceptos

1. Describa cómo los lenguajes orientados a objetos brindan polimorfismo y son a la vez fuertemente tipados. ¿Es necesario realizar todos los chequeos de tipo en tiempo de compilación?
2. Explique por qué en los lenguajes orientados a objetos es necesario que la ligadura entre un mensaje (llamada) y el método correspondiente se resuelva dinámicamente.
3. Analice la noción de efectos colaterales en el contexto de la programación orientada a objetos. ¿Es posible evitarlos? Justifique su respuesta.
4. Analice bajo qué circunstancias la herencia múltiple puede presentar problemas. ¿Qué mecanismos pueden implementarse para evitarlos?
5. En algunos problemas, una clasificación ortogonal de clases resulta más natural. Por ejemplo, suponga que se desea agrupar las distintas especies según su tipo (ave, mamífero, reptil, etc.) y sus hábitos alimenticios (herbívoros o carnívoros). De este modo, la clasificación puede disponerse según el siguiente cuadro:

	Ave	Mamífero	Reptil
Herbívoros	Gallina, Urraca, etc.	Ciervo, Conejo, etc.	Tortuga Terrestre
Carnívoros	Águila, Halcón, etc.	León, Zorros, etc.	Víbora, Caimán, etc.

- Observando la tabla anterior, vemos que un león es un mamífero carnívoro, luego tendrá los atributos y comportamiento de un Mamífero más los de un Carnívoro. En su opinión, ¿qué forma de herencia basada en clases se ajusta mejor al modelado de este problema?
 - Por otra parte, también existen especies que aun siendo mamíferos (ej. el ornitorrinco) poseen características propias de los reptiles. ¿Cómo incorporaría esta categoría dentro de la clasificación anterior? ¿Qué forma de herencia basada en clases resulta más apropiada para este caso?
6. Analice la siguiente jerarquía de clases, asumiendo herencia simple:



De la clasificación anterior se aprecia que todas las instancias de las clases Figura Geométrica, Circunferencia, Elipse, Rectángulo, Cuadrado y Triángulo poseen en su protocolo las operaciones: escalar, rotar y área. Dentro de cada clase se indica las operaciones definidas. Por ejemplo, escalar es definida en Figura Geométrica y redefinida en Rectángulo y Triángulo. Por otra parte, área está definida en Figura Geométrica y redefinida en Circunferencia, Elipse, Rectángulo y Cuadrado. La pregunta es: ¿son polimórficas las operaciones escalar, rotar y área? Justifique sus afirmaciones.

Casos de Estudio e Investigación

7. Retomando lo realizado en el ejercicio 1, analice si Smalltalk es un lenguaje fuertemente tipado.
8. Evalúe en el intérprete de GNU-Smalltalk las siguientes expresiones:

- a. `3 >= 8 and: [ 2 <= 4 ]`
- b. `'comenzamos con Smalltalk' size`
- c. `#( 1 3 5 7 ) at: 2`
- d. `'Buenos' , ' días'`
- e. `'Buenos días' at: 8`
- f. `[3+1.5*3] value`
- g. `[ :i :j | 3*i - (i-j)] value: 4 value: 2`
- h. `#( 1 12 24 36) includes: 4 factorial`
- i. `'smalltalk' at: (#( 5 3 1 ) at: 2 )`
- j. `'smalltalk' at: #( 5 3 1 ) at: 2`
- k. `'Lenguaje Orientados a Objetos' copyFrom: 10 to: 19; size`
- l. `('Lenguaje Orientados a Objetos' copyFrom: 10 to: 19) size`
- m. `4 factorial between: 3 + 4 and: 'hello' size * 7`
- n. `3 < 4 ifTrue: ['the true block'] ifFalse: ['the false block']`

Para cada mensaje, identifique las clases de los objetos receptores. Para el caso de los mensajes que incluyen otros mensajes, determine en qué orden se efectúan las evaluaciones. Puede hacer uso de los mensajes `print` o `printNl` para imprimir los resultados por pantalla.

9. Evalúe en GNU-Smalltalk las siguientes secuencias de expresiones y explique lo que sucede en cada caso:

<pre> max a b a := 5 squared. b := 4 factorial. a < b ifTrue: [max := b] ifFalse: [max := a]. max print</pre>	<pre> t i facs facs := {3. 4. 5. 6}. i := 1. facs size timesRepeat:[t:= facs at: i. facs at:i put:t factorial. i:= i + 1]. facs print</pre>
---	---

10. Considere las siguientes clases definidas en GNU-Smalltalk. Sea `z` un objeto de la clase `B`, indique el valor que se obtiene de la ejecución de los siguientes mensajes:

- a. `z met2`
- b. `z met3`

<pre>Object subclass: A [i j A class >> new [^(super new) inicializar] inicializar [i := 2. j := 4] met1 [^(i+j)]]</pre>	<pre>A subclass: B [inicializar [i := 3. j := 6] met1 [^(i*j)] met2 [^(self met1)] met3 [^(super met1)]]</pre>
--	--

11. Extienda la definición de la clase `String` de Smalltalk con los siguientes métodos (para recorrer el string debe utilizar un iterador):

- a. *cantDigitos*, retorna la cantidad de dígitos presentes en la cadena receptora del mensaje.
- b. *letras*, retorna el conjunto de letras presentes en la cadena receptora del mensaje.

12. Dada la declaración de las siguientes clases:

```
Object subclass: A [  
  |va|  
  A class >> new [  
    ^(super new) inicializar  
  ]  
  inicializar [ va:=1 ]  
  va [^va]  
  va: valor [^(va:=valor)]  
]
```

```
A subclass: B [  
  |vb|  
  A class >> new [  
    ^(super new) inicializar  
  ]  
  inicializar [ va:=1. vb:=10 ]  
  vb [^vb]  
]
```

Evalúe en el intérprete de GNU-Smalltalk cada uno de los siguientes mensajes, y explique los conceptos involucrados en cada caso. En el caso de que alguno de los mensajes dé error, vuelva a evaluar eliminando ese mensaje y explique la causa del error.

```
| a b c |  
a := A new. a va.  
b := B new. b vb. b va.  
c:= B new. c:=a. c va. c vb. a va: 10. c va.
```

13. Investigue sobre los distintos iteradores soportados por Smalltalk. ¿En qué clases están definidos? ¿Cómo se utilizan? Compárelos con los mecanismos de iteración provistos por Python y Java.
14. Agregue al conjunto de métodos de la clase BlockClosure de Smalltalk una nueva estructura de control similar al repeat until de Pascal. A modo de ejemplo, si evaluamos la siguiente secuencia de expresiones, el valor de p al finalizar es el objeto 10:

```
| p k val |  
p := 0.  
k := 0.  
[k := k+1. p := p+k] repeatUntil: [p > 8].
```

15. Extienda la clase Array de Smalltalk con el método *rotar*:, el cual rota los elementos del arreglo hacia la derecha, tantas veces como lo indique el argumento. Por ejemplo, sea *arreglo1* el siguiente arreglo: [1 2 3 4 5 6], después de ejecutarse el mensaje *arreglo1 rotar: 3*, *arreglo1* queda como sigue: [4 5 6 1 2 3].
16. La implementación del método *and*: en Smalltalk es la siguiente:

```
Boolean subclass: True [  
  ...  
  and: aBlock [^aBlock value]  
]
```

```
Boolean subclass: False [  
  ...  
  and: aBlock [^false]  
]
```

- a. Explique cómo funciona este método (puede hacerlo con ejemplos).
- b. Relacione la implementación del *and*: en Smalltalk con los órdenes de evaluación vistos en la materia.
- c. ¿El operador binario & se comporta de la misma manera? Muestre un ejemplo que sirva para justificar su respuesta.
17. Analice el soporte para el concepto de polimorfismo provisto por Smalltalk. Como parte de su análisis, considere el método *and*: del ejercicio anterior.
18. ¿Qué diferencias ve entre el sistema de tipos de Java y el Smalltalk? Realice un análisis comparativo entre estos lenguajes utilizando los criterios y/o características vistos en la materia.
19. Compare la forma en que Smalltalk modela los tipos que son considerados primitivos en Java.
20. ¿Qué diferencias ve en la semántica de los operadores aritméticos en Smalltalk con respecto a otros lenguajes orientados a objetos como Java? ¿Qué ventajas y desventajas tienen asociadas esas diferencias?
21. Utilice algún Iterador para recorrer el objeto receptor en cada uno de los siguientes incisos:

- a. Extienda la clase `Array` con el método *calcular*:. Este método tiene una funcionalidad similar a la del iterador *collect*:, ya que retorna un arreglo cuyos elementos son el resultado de evaluar el bloque recibido como argumento, para cada uno de los elementos del arreglo receptor. Por ejemplo:

```
#(4 1 2 3) calcular: [:i | i even.] → #(true false true false)
#(4 1 2 3) calcular: [:i | i + 10.] → #(14 11 12 13)
#('hola' 1 'no' 3.0) calcular: [:i | i isInteger.] → #(false true false false)
```

- b. Extienda la clase `Collection` con el método *incluye*:. Este método determina si el receptor contiene algún elemento igual a un objeto recibido como argumento. Por ejemplo:

```
#(3 'si' 1 2) incluye: 'si' → true
'murcielago' incluye: $u → $\rightarrow$ true
#(2.0 'no' 2 $a) incluye: 4 → false
```

- c. Extienda la clase `String` con el método *soloLetras*. Este método debe retornar un nuevo string conteniendo solo las letras presentes en la cadena receptora. Por ejemplo:

```
'Tengo 30 años' soloLetras → 'Tengoanios'
```